

Process Verification

Lab 7

Quentin Meneghini

22nd May 2026

Facultat d'Informàtica de Barcelona

Contents

1	Design Overview	1
1.1	Overview	1
1.2	Module Interface	1
1.3	State Machine Design	1
1.4	Formal Verification (SVA)	3
2	Plan	4
3	Formal Statements	5
3.1	Logic	5
3.2	Assumptions	5
3.3	Assertions	6
4	Results	9
4.1	Base	9
4.2	Bug 0	9
4.3	Bug 1	10
A	Appendix Chapter	11
A.1	Use of tools	11
A.2	Use of time	11
A.3	Use of AI	11
A.4	Output Valid	12
A.5	Output Bugged 0	13
A.6	Output Bugged 1	18

Chapter 1

Design Overview

1.1 Overview

The `packet` module implements a synchronous Finite State Machine (FSM) that models the reception of a data packet through successive processing stages: header, payload, checksum, and completion. It generates status flags (`valid_pkt`, `error_pkt`) to indicate whether a packet was successfully received or failed due to checksum errors.

1.2 Module Interface

Signal Name	Direction	Description
<code>clk</code>	Input	System clock. All state transitions occur on the rising edge.
<code>reset</code>	Input	Synchronous active-high reset. Forces the FSM to IDLE.
<code>start_pkt</code>	Input	Initiates packet reception (valid in IDLE).
<code>hdr_done</code>	Input	Signals completion of header processing.
<code>payload_done</code>	Input	Signals completion of payload processing.
<code>chk_ok</code>	Input	Indicates checksum verification passed.
<code>chk_fail</code>	Input	Indicates checksum verification failed.
<code>abort</code>	Input	Aborts reception at any stage, forcing FSM to IDLE.
<code>valid_pkt</code>	Output	Pulses high when a packet is successfully received.
<code>error_pkt</code>	Output	Pulses high when checksum fails.
<code>state[2:0]</code>	Output	Current encoded state of the FSM.

1.3 State Machine Design

The FSM consists of five states. An asynchronous `abort` signal can transition the system back to IDLE from any active state.

State Diagram

Every nameless arrow implies that all inputs that have not been specified translate in that state.

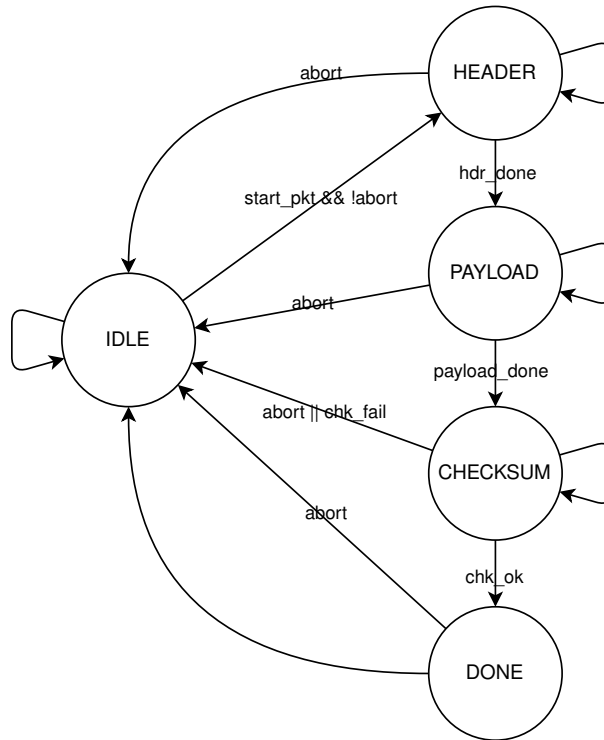


Figure 1.1: svg image

State Descriptions

- **IDLE (3'b000):** The default state. Waits for `start_pkt` to begin processing. Outputs are cleared.
- **HEADER (3'b001):** Processing the packet header. Holds state until `hdr_done` is asserted.
- **PAYLOAD (3'b010):** Processing the packet payload. Holds state until `payload_done` is asserted.
- **CHECKSUM (3'b011):** Verifying data integrity. Branches based on `chk_ok` or `chk_fail`. Assumes both cannot be asserted simultaneously.
- **DONE (3'b100):** Asserts `valid_pkt` and automatically returns to IDLE on the next clock cycle.

1.4 Formal Verification (SVA)

To ensure the robustness of the RTL implementation, the design has been bounded-model checked using **EBMC** and SystemVerilog Assertions (SVA). Key formal properties include:

1. **State Reachability & Hold:** Guarantees that the FSM only transitions on valid signals and holds its state otherwise.
2. **Spurious Output Prevention:** Uses the `$past()` operator to ensure `valid_pkt` and `error_pkt` only pulse high immediately following the correct checksum evaluation.
3. **Reset & Abort Compliance:** Proves that a reset or abort strictly clears outputs and returns the state space to IDLE, overriding any active transitions.

Chapter 2

Plan

- Understand the dut functionality and specification
- Determine the finite state machine graph ($\text{input} \rightarrow \text{state}$)
- Determine how the graph translates into outputs ($\text{state} \rightarrow \text{output}$)
- Determine assumption on inputs
- Determine edge cases
- Determine timing issues
- Determine assertions on outputs
- Implementation
- Debugging

Chapter 3

Formal Statements

Here we will introduce some useful logic followed by each statement, starting from the assumptions and then move on to the assertions.

3.1 Logic

past_valid A helper register initialized to 0 that transitions to 1 after the first clock edge. It safely gates properties that evaluate historical data via the `$past()` operator, preventing tool errors on cycle zero.

```
58 logic past_valid = 0'b0;  
59 always @(posedge clk) past_valid <= 1'b1;
```

3.2 Assumptions

initial_reset Forces the `reset` signal to be active during the very first clock cycle (when `past_valid` is 0) to ensure the FSM initializes cleanly.

```
61 initial_reset: assume property (  
62     @(posedge clk) !past_valid |-> reset  
63 );
```

exclusive_outcome Constrains the solver so that `chk_ok` and `chk_fail` are mutually exclusive and never asserted simultaneously. This could have been transformed into an assertion, but it would have tested the input instead of the dut itself.

```
65 exclusive_outcome: assume property (  
66     @(posedge clk) !chk_ok || !chk_fail  
67 );
```

3.3 Assertions

_reset & _abort Verify that asserting a reset or an abort unconditionally forces the FSM back to the IDLE state on the next cycle with both output flags cleared.

```

69  _reset: assert property (
70      @(posedge clk)
71      reset | => (state == IDLE) && !valid_pkt && !error_pkt
72  );
73
74  _abort: assert property (
75      @(posedge clk) disable iff (reset)
76      abort | => (state == IDLE) && !valid_pkt && !error_pkt
77  );

```

_state Guarantees the FSM is mathematically restricted to the five valid encoded states, proving no illegal states can be reached.

```

79  _state: assert property (
80      @(posedge clk) disable iff (reset)
81      (state == IDLE) ||
82      (state == HEADER) ||
83      (state == PAYLOAD) ||
84      (state == CHECKSUM) ||
85      (state == DONE)
86  );

```

State Hold (idle_idle, header_header, payload_payload, checksum_checksum): Guarantee that the FSM retains its current state when no progression signal or abort is asserted.

```

124  idle_idle: assert property (
125      @(posedge clk) disable iff (reset)
126      (state == IDLE) && !start_pkt && !abort
127      | => (state == IDLE)
128  );
129
130  header_header: assert property (
131      @(posedge clk) disable iff (reset)
132      (state == HEADER) && !hdr_done && !abort
133      | => (state == HEADER)
134  );
135
136  payload_payload: assert property (
137      @(posedge clk) disable iff (reset)
138      (state == PAYLOAD) && !payload_done && !abort
139      | => (state == PAYLOAD)
140  );
141
142  checksum_checksum: assert property (
143      @(posedge clk) disable iff (reset)
144      (state == CHECKSUM) && !chk_ok && !chk_fail && !abort
145      | => (state == CHECKSUM) && !valid_pkt && !error_pkt
146  );

```

State Progression (`idle_header`, `header_payload`, `payload_checksum`, `checksum_done`, `checksum_idle`, `done_idle`): Prove that the FSM correctly transitions to the next expected state when the appropriate progression flag is asserted.

```

88     idle_header: assert property (
89         @(posedge clk) disable iff (reset)
90         (state == IDLE) && start_pkt && !abort
91         |> (state == HEADER)
92     );
93
94     header_payload: assert property (
95         @(posedge clk) disable iff (reset)
96         (state == HEADER) && hdr_done && !abort
97         |> (state == PAYLOAD)
98     );
99
100    payload_checksum: assert property (
101        @(posedge clk) disable iff (reset)
102        (state == PAYLOAD) && payload_done && !abort
103        |> (state == CHECKSUM)
104    );
105
106    checksum_done: assert property (
107        @(posedge clk) disable iff (reset)
108        (state == CHECKSUM) && chk_ok && !abort
109        |> (state == DONE) && valid_pkt
110    );
111
112    checksum_idle: assert property (
113        @(posedge clk) disable iff (reset)
114        ((state == CHECKSUM) && chk_fail && !abort)
115        |> (state == IDLE) && error_pkt
116    );
117
118    done_idle: assert property (
119        @(posedge clk) disable iff (reset)
120        (state == DONE) && !abort
121        |> (state == IDLE)
122    );

```

Output Safety (`safe_valid_pkt`, `safe_error_pkt`): Prove that the output flags are never asserted spuriously. They only pulse high if the FSM was actively in the `CHECKSUM` state during the previous cycle with the correct outcome flag.

```

148    safe_error_pkt: assert property (
149        @(posedge clk) disable iff (reset)
150        past_valid && error_pkt
151        |> ($past(state) == CHECKSUM) &&
152           $past(chk_fail) &&
153           !$past(abort)
154    );
155
156    safe_valid_pkt: assert property (
157        @(posedge clk) disable iff (reset)
158        past_valid && valid_pkt
159        |> ($past(state) == CHECKSUM) &&
160           $past(chk_ok) &&
161           !$past(abort)
162    );

```

Output Pulse (pulse_valid_pkt, pulse_error_pkt): Verify that the output flags remain asserted for exactly one clock cycle, preventing it from a pulse behavior.

```
164     pulse_error_pkt: assert property (  
165         @(posedge clk) disable iff (reset)  
166         past_valid && error_pkt  
167         |-> !$past(error_pkt)  
168     );  
169  
170     pulse_valid_pkt: assert property (  
171         @(posedge clk) disable iff (reset)  
172         past_valid && valid_pkt  
173         |-> !$past(valid_pkt)  
174     );
```

Chapter 4

Results

Here we will test the base architecture, and introduce some bugs to test failure as well. The complete outputs can be retrieved in the appendix and will be linked promptly.

Experiment	Injected Bug	Failed Properties	Counterexamples
Base	None	None	None
Bug 0	DONE state instead of IDLE	<code>checksum_idle</code>	1
Bug 1	Invalid state 3'd5 instead of IDLE	<code>_abort</code> , <code>_state</code>	2

Table 4.1: Summary of bugs and resulting property violations

4.1 Base

The execution is flawless and very fast.

The trace can be reviewed in the appendix A.4.

4.2 Bug 0

We used the DONE state instead of IDLE after a failing packet.

Original: `state <= IDLE;`

Bugged: `state <= DONE;`

The trace and the waveform can be reviewed in the appendix A.5.

Analysis

The tool **REFUTE** the `checksum_idle` property. A single counter-example is given and it shows the trace that leads to a failure of the assertion. At state 10 the FSM is in the state `CHECKSUM` (011) with `chk_fail` asserted, but instead of returning to `IDLE` (000), the bug forces the FSM to become `DONE` (100).

4.3 Bug 1

Here we changed the `abort` state into a non-existent one: `3'd5`.

Original: `state <= IDLE;`

Bugged: `state <= 3'd5;`

The trace and the waveform can be reviewed in the appendix A.6.

Analysis

The tool **REFUTE** two separate properties simultaneously: `_abort` and `_state`. Two distinct counterexample traces were generated, one for each property, but they both look at the same bug from different perspectives. In state 9 the FSM is in the unknown state (101), this is a violation of both properties. On one hand, the inputs that led the dut to the violation, should have returned a `DONE` state. On the other hand, that state does not even exist.

Appendix A

Appendix Chapter

A.1 Use of tools

- Visual Studio Code: systemverilog IDE.
- QuestaSim: CLI tester.
- GTKwave: wave visualizer.

A.2 Use of time

A total of 8 h distributed as follows.

- Understanding the problem: 1h.
- Planning: 1h.
- Implementation: 1h.
- Bugs: 3h.
- Report: 2h.

A.3 Use of AI

- Gemini (Pro): sentence enhancing, bug fixer

A.4 Output Valid

```

0 Parsing packet.sv
1 Converting
2 Type-checking Verilog::packet
3 Generating Decision Problem
4 Using MiniSAT 2.2.1 with simplifier
5 Properties
6 Solving with propositional reduction
7 SAT checker: instance is UNSATISFIABLE
8 UNSAT: No path found within bound
9
10 ** Results:
11 [packet.initial_reset] always (!packet.past_valid |-> packet.reset): ASSUMED
12 [packet.exclusive_outcome] always !packet.chk_ok || !packet.chk_fail: ASSUMED
13 [packet._reset] always (packet.reset |>= packet.state == packet.IDLE && !packet
14   .valid_pkt && !packet.error_pkt): PROVED up to bound 20
15 [packet._abort] always (disable iff (packet.reset) packet.abort |>= packet.
16   state == packet.IDLE && !packet.valid_pkt && !packet.error_pkt): PROVED up
17   to bound 20
18 [packet._state] always (disable iff (packet.reset) packet.state == packet.IDLE
19   || packet.state == packet.HEADER || packet.state == packet.PAYLOAD || packet
20   .state == packet.CHECKSUM || packet.state == packet.DONE): PROVED up to
21   bound 20
22 [packet.idle_header] always (disable iff (packet.reset) packet.state == packet.
23   IDLE && packet.start_pkt && !packet.abort |>= packet.state == packet.HEADER)
24   : PROVED up to bound 20
25 [packet.idle_idle] always (disable iff (packet.reset) packet.state == packet.
26   IDLE && !packet.start_pkt && !packet.abort |>= packet.state == packet.IDLE):
27   PROVED up to bound 20
28 [packet.header_payload] always (disable iff (packet.reset) packet.state ==
29   packet.HEADER && packet.hdr_done && !packet.abort |>= packet.state == packet
30   .PAYLOAD): PROVED up to bound 20
31 [packet.header_header] always (disable iff (packet.reset) packet.state ==
32   packet.HEADER && !packet.hdr_done && !packet.abort |>= packet.state ==
33   packet.HEADER): PROVED up to bound 20
34 [packet.payload_checksum] always (disable iff (packet.reset) packet.state ==
35   packet.PAYLOAD && packet.payload_done && !packet.abort |>= packet.state ==
36   packet.CHECKSUM): PROVED up to bound 20
37 [packet.payload_payload] always (disable iff (packet.reset) packet.state ==
38   packet.PAYLOAD && !packet.payload_done && !packet.abort |>= packet.state ==
39   packet.PAYLOAD): PROVED up to bound 20
40 [packet.checksum_done] always (disable iff (packet.reset) packet.state ==
41   packet.CHECKSUM && packet.chk_ok && !packet.abort |>= packet.state == packet
42   .DONE && packet.valid_pkt): PROVED up to bound 20
43 [packet.checksum_idle] always (disable iff (packet.reset) packet.state ==
44   packet.CHECKSUM && packet.chk_fail && !packet.abort |>= packet.state ==
45   packet.IDLE && packet.error_pkt): PROVED up to bound 20
46 [packet.checksum_checksum] always (disable iff (packet.reset) packet.state ==
47   packet.CHECKSUM && !packet.chk_ok && !packet.chk_fail && !packet.abort |>=
48   packet.state == packet.CHECKSUM && !packet.valid_pkt && !packet.error_pkt):
49   PROVED up to bound 20
50 [packet.done_idle] always (disable iff (packet.reset) packet.state == packet.
51   DONE && !packet.abort |>= packet.state == packet.IDLE): PROVED up to bound
52   20
53 [packet.pulse_error_pkt] always (disable iff (packet.reset) packet.past_valid
54   && packet.error_pkt |-> !$past(packet.error_pkt)): PROVED up to bound 20
55 [packet.safe_error_pkt] always (disable iff (packet.reset) packet.past_valid &&
56   packet.error_pkt |-> $past(packet.state) == packet.CHECKSUM && $past(packet
57   .chk_fail) && !$past(packet.abort)): PROVED up to bound 20
58 [packet.pulse_valid_pkt] always (disable iff (packet.reset) packet.past_valid
59   && packet.valid_pkt |-> !$past(packet.valid_pkt)): PROVED up to bound 20

```

```

29 [packet.safe_valid_pkt] always (disable iff (packet.reset) packet.past_valid &&
    packet.valid_pkt |-> $past(packet.state) == packet.CHECKSUM && $past(packet
    .chk_ok) && !$past(packet.abort)): PROVED up to bound 20

```

A.5 Output Bugged 0

```

0 Parsing packet.sv
1 Converting
2 Type-checking Verilog::packet
3 Generating Decision Problem
4 Using MiniSAT 2.2.1 with simplifier
5 Properties
6 Solving with propositional reduction
7 SAT checker: instance is SATISFIABLE
8 SAT: path found
9 SAT checker: instance is UNSATISFIABLE
10 UNSAT: No path found within bound
11
12 ** Results:
13 [packet.initial_reset] always (!packet.past_valid |-> packet.reset): ASSUMED
14 [packet.exclusive_outcome] always !packet.chk_ok || !packet.chk_fail: ASSUMED
15 [packet._reset] always (packet.reset |=> packet.state == packet.IDLE && !packet
    .valid_pkt && !packet.error_pkt): PROVED up to bound 20
16 [packet._abort] always (disable iff (packet.reset) packet.abort |=> packet.
    state == packet.IDLE && !packet.valid_pkt && !packet.error_pkt): PROVED up
    to bound 20
17 [packet._state] always (disable iff (packet.reset) packet.state == packet.IDLE
    || packet.state == packet.HEADER || packet.state == packet.PAYLOAD || packet
    .state == packet.CHECKSUM || packet.state == packet.DONE): PROVED up to
    bound 20
18 [packet.idle_header] always (disable iff (packet.reset) packet.state == packet.
    IDLE && packet.start_pkt && !packet.abort |=> packet.state == packet.HEADER)
    : PROVED up to bound 20
19 [packet.idle_idle] always (disable iff (packet.reset) packet.state == packet.
    IDLE && !packet.start_pkt && !packet.abort |=> packet.state == packet.IDLE):
    PROVED up to bound 20
20 [packet.header_payload] always (disable iff (packet.reset) packet.state ==
    packet.HEADER && packet.hdr_done && !packet.abort |=> packet.state == packet
    .PAYLOAD): PROVED up to bound 20
21 [packet.header_header] always (disable iff (packet.reset) packet.state ==
    packet.HEADER && !packet.hdr_done && !packet.abort |=> packet.state ==
    packet.HEADER): PROVED up to bound 20
22 [packet.payload_checksum] always (disable iff (packet.reset) packet.state ==
    packet.PAYLOAD && packet.payload_done && !packet.abort |=> packet.state ==
    packet.CHECKSUM): PROVED up to bound 20
23 [packet.payload_payload] always (disable iff (packet.reset) packet.state ==
    packet.PAYLOAD && !packet.payload_done && !packet.abort |=> packet.state ==
    packet.PAYLOAD): PROVED up to bound 20
24 [packet.checksum_done] always (disable iff (packet.reset) packet.state ==
    packet.CHECKSUM && packet.chk_ok && !packet.abort |=> packet.state == packet
    .DONE && packet.valid_pkt): PROVED up to bound 20
25 [packet.checksum_idle] always (disable iff (packet.reset) packet.state ==
    packet.CHECKSUM && packet.chk_fail && !packet.abort |=> packet.state ==
    packet.IDLE && packet.error_pkt): REFUTED
26 Counterexample:
27
28 Transition system state 0
29 -----
30 packet.clk = ?
31 packet.reset = 1
32 packet.start_pkt = 1

```

```

33 packet.hdr_done = 1
34 packet.payload_done = 0
35 packet.chk_ok = 0
36 packet.chk_fail = 1
37 packet.abort = 0
38 packet.valid_pkt = 0
39 packet.error_pkt = 0
40 packet.state = 3'b111 (111)
41 packet.IDLE = 3'b000 (000)
42 packet.HEADER = 3'b001 (001)
43 packet.PAYLOAD = 3'b010 (010)
44 packet.CHECKSUM = 3'b011 (011)
45 packet.DONE = 3'b100 (100)
46 packet.past_valid = 1'b0
47
48 Transition system state 1
49 -----
50 packet.clk = ?
51 packet.reset = 0
52 packet.start_pkt = 1
53 packet.hdr_done = 0
54 packet.payload_done = 0
55 packet.chk_ok = 0
56 packet.chk_fail = 0
57 packet.abort = 0
58 packet.valid_pkt = 0
59 packet.error_pkt = 0
60 packet.state = 3'b000 (000)
61 packet.IDLE = 3'b000 (000)
62 packet.HEADER = 3'b001 (001)
63 packet.PAYLOAD = 3'b010 (010)
64 packet.CHECKSUM = 3'b011 (011)
65 packet.DONE = 3'b100 (100)
66 packet.past_valid = 1'b1
67
68 Transition system state 2
69 -----
70 packet.clk = ?
71 packet.reset = 0
72 packet.start_pkt = 0
73 packet.hdr_done = 1
74 packet.payload_done = 0
75 packet.chk_ok = 0
76 packet.chk_fail = 1
77 packet.abort = 1
78 packet.valid_pkt = 0
79 packet.error_pkt = 0
80 packet.state = 3'b001 (001)
81 packet.IDLE = 3'b000 (000)
82 packet.HEADER = 3'b001 (001)
83 packet.PAYLOAD = 3'b010 (010)
84 packet.CHECKSUM = 3'b011 (011)
85 packet.DONE = 3'b100 (100)
86 packet.past_valid = 1'b1
87
88 Transition system state 3
89 -----
90 packet.clk = ?
91 packet.reset = 0
92 packet.start_pkt = 1
93 packet.hdr_done = 0
94 packet.payload_done = 0
95 packet.chk_ok = 1

```

```

96     packet.chk_fail = 0
97     packet.abort = 0
98     packet.valid_pkt = 0
99     packet.error_pkt = 0
100    packet.state = 3'b000 (000)
101    packet.IDLE = 3'b000 (000)
102    packet.HEADER = 3'b001 (001)
103    packet.PAYLOAD = 3'b010 (010)
104    packet.CHECKSUM = 3'b011 (011)
105    packet.DONE = 3'b100 (100)
106    packet.past_valid = 1'b1
107
108    Transition system state 4
109    -----
110    packet.clk = ?
111    packet.reset = 0
112    packet.start_pkt = 0
113    packet.hdr_done = 1
114    packet.payload_done = 0
115    packet.chk_ok = 0
116    packet.chk_fail = 0
117    packet.abort = 0
118    packet.valid_pkt = 0
119    packet.error_pkt = 0
120    packet.state = 3'b001 (001)
121    packet.IDLE = 3'b000 (000)
122    packet.HEADER = 3'b001 (001)
123    packet.PAYLOAD = 3'b010 (010)
124    packet.CHECKSUM = 3'b011 (011)
125    packet.DONE = 3'b100 (100)
126    packet.past_valid = 1'b1
127
128    Transition system state 5
129    -----
130    packet.clk = ?
131    packet.reset = 0
132    packet.start_pkt = 0
133    packet.hdr_done = 0
134    packet.payload_done = 0
135    packet.chk_ok = 0
136    packet.chk_fail = 0
137    packet.abort = 0
138    packet.valid_pkt = 0
139    packet.error_pkt = 0
140    packet.state = 3'b010 (010)
141    packet.IDLE = 3'b000 (000)
142    packet.HEADER = 3'b001 (001)
143    packet.PAYLOAD = 3'b010 (010)
144    packet.CHECKSUM = 3'b011 (011)
145    packet.DONE = 3'b100 (100)
146    packet.past_valid = 1'b1
147
148    Transition system state 6
149    -----
150    packet.clk = ?
151    packet.reset = 0
152    packet.start_pkt = 0
153    packet.hdr_done = 0
154    packet.payload_done = 1
155    packet.chk_ok = 0
156    packet.chk_fail = 1
157    packet.abort = 0
158    packet.valid_pkt = 0

```

```

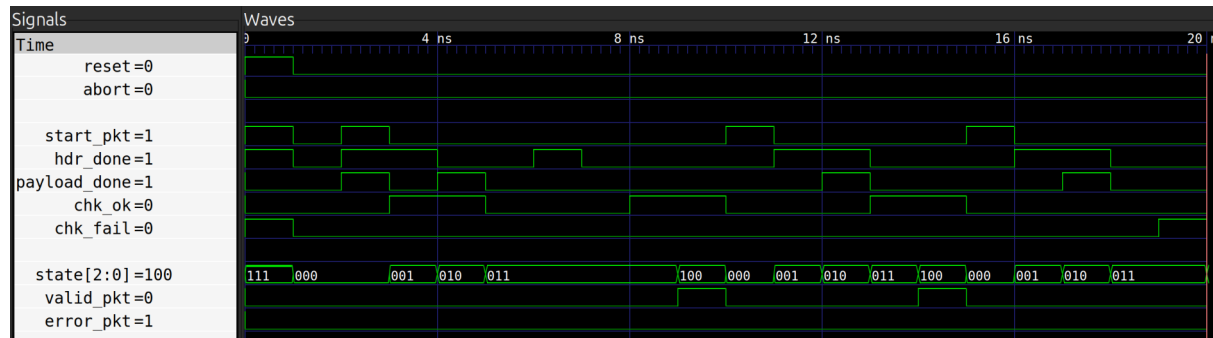
159     packet.error_pkt = 0
160     packet.state = 3'b010 (010)
161     packet.IDLE = 3'b000 (000)
162     packet.HEADER = 3'b001 (001)
163     packet.PAYLOAD = 3'b010 (010)
164     packet.CHECKSUM = 3'b011 (011)
165     packet.DONE = 3'b100 (100)
166     packet.past_valid = 1'b1
167
168 Transition system state 7
169 -----
170     packet.clk = ?
171     packet.reset = 0
172     packet.start_pkt = 0
173     packet.hdr_done = 1
174     packet.payload_done = 0
175     packet.chk_ok = 0
176     packet.chk_fail = 0
177     packet.abort = 0
178     packet.valid_pkt = 0
179     packet.error_pkt = 0
180     packet.state = 3'b011 (011)
181     packet.IDLE = 3'b000 (000)
182     packet.HEADER = 3'b001 (001)
183     packet.PAYLOAD = 3'b010 (010)
184     packet.CHECKSUM = 3'b011 (011)
185     packet.DONE = 3'b100 (100)
186     packet.past_valid = 1'b1
187
188 Transition system state 8
189 -----
190     packet.clk = ?
191     packet.reset = 0
192     packet.start_pkt = 0
193     packet.hdr_done = 0
194     packet.payload_done = 0
195     packet.chk_ok = 0
196     packet.chk_fail = 0
197     packet.abort = 0
198     packet.valid_pkt = 0
199     packet.error_pkt = 0
200     packet.state = 3'b011 (011)
201     packet.IDLE = 3'b000 (000)
202     packet.HEADER = 3'b001 (001)
203     packet.PAYLOAD = 3'b010 (010)
204     packet.CHECKSUM = 3'b011 (011)
205     packet.DONE = 3'b100 (100)
206     packet.past_valid = 1'b1
207
208 Transition system state 9
209 -----
210     packet.clk = ?
211     packet.reset = 0
212     packet.start_pkt = 0
213     packet.hdr_done = 0
214     packet.payload_done = 0
215     packet.chk_ok = 0
216     packet.chk_fail = 0
217     packet.abort = 0
218     packet.valid_pkt = 0
219     packet.error_pkt = 0
220     packet.state = 3'b011 (011)
221     packet.IDLE = 3'b000 (000)

```

```

222 packet.HEADER = 3'b001 (001)
223 packet.PAYLOAD = 3'b010 (010)
224 packet.CHECKSUM = 3'b011 (011)
225 packet.DONE = 3'b100 (100)
226 packet.past_valid = 1'b1
227
228 Transition system state 10
229 -----
230 packet.clk = ?
231 packet.reset = 0
232 packet.start_pkt = 0
233 packet.hdr_done = 0
234 packet.payload_done = 0
235 packet.chk_ok = 0
236 packet.chk_fail = 1
237 packet.abort = 0
238 packet.valid_pkt = 0
239 packet.error_pkt = 0
240 packet.state = 3'b011 (011)
241 packet.IDLE = 3'b000 (000)
242 packet.HEADER = 3'b001 (001)
243 packet.PAYLOAD = 3'b010 (010)
244 packet.CHECKSUM = 3'b011 (011)
245 packet.DONE = 3'b100 (100)
246 packet.past_valid = 1'b1
247
248 Transition system state 11
249 -----
250 packet.clk = ?
251 packet.reset = 0
252 packet.start_pkt = 0
253 packet.hdr_done = 1
254 packet.payload_done = 0
255 packet.chk_ok = 0
256 packet.chk_fail = 0
257 packet.abort = 0
258 packet.valid_pkt = 0
259 packet.error_pkt = 1
260 packet.state = 3'b100 (100)
261 packet.IDLE = 3'b000 (000)
262 packet.HEADER = 3'b001 (001)
263 packet.PAYLOAD = 3'b010 (010)
264 packet.CHECKSUM = 3'b011 (011)
265 packet.DONE = 3'b100 (100)
266 packet.past_valid = 1'b1
267
268 [packet.checksum_checksum] always (disable iff (packet.reset) packet.state ==
    packet.CHECKSUM && !packet.chk_ok && !packet.chk_fail && !packet.abort | =>
    packet.state == packet.CHECKSUM && !packet.valid_pkt && !packet.error_pkt):
    PROVED up to bound 20
269 [packet.done_idle] always (disable iff (packet.reset) packet.state == packet.
    DONE && !packet.abort | => packet.state == packet.IDLE): PROVED up to bound
    20
270 [packet.pulse_error_pkt] always (disable iff (packet.reset) packet.past_valid
    && packet.error_pkt | -> !$past(packet.error_pkt)): PROVED up to bound 20
271 [packet.safe_error_pkt] always (disable iff (packet.reset) packet.past_valid &&
    packet.error_pkt | -> $past(packet.state) == packet.CHECKSUM && $past(packet
    .chk_fail) && !$past(packet.abort)): PROVED up to bound 20
272 [packet.pulse_valid_pkt] always (disable iff (packet.reset) packet.past_valid
    && packet.valid_pkt | -> !$past(packet.valid_pkt)): PROVED up to bound 20
273 [packet.safe_valid_pkt] always (disable iff (packet.reset) packet.past_valid &&
    packet.valid_pkt | -> $past(packet.state) == packet.CHECKSUM && $past(packet
    .chk_ok) && !$past(packet.abort)): PROVED up to bound 20

```



A.6 Output Bugged 1

```

0 Parsing packet.sv
1 Converting
2 Type-checking Verilog::packet
3 Generating Decision Problem
4 Using MiniSAT 2.2.1 with simplifier
5 Properties
6 Solving with propositional reduction
7 SAT checker: instance is SATISFIABLE
8 SAT: path found
9 SAT checker: instance is SATISFIABLE
10 SAT: path found
11 SAT checker: instance is UNSATISFIABLE
12 UNSAT: No path found within bound
13
14 ** Results:
15 [packet.initial_reset] always (!packet.past_valid |-> packet.reset): ASSUMED
16 [packet.exclusive_outcome] always !packet.chk_ok || !packet.chk_fail: ASSUMED
17 [packet._reset] always (packet.reset |>= packet.state == packet.IDLE && !packet
    .valid_pkt && !packet.error_pkt): PROVED up to bound 20
18 [packet._abort] always (disable iff (packet.reset) packet.abort |>= packet.
    state == packet.IDLE && !packet.valid_pkt && !packet.error_pkt): REFUTED
19 Counterexample:
20
21 Transition system state 0
22 -----
23 packet.clk = ?
24 packet.reset = 1
25 packet.start_pkt = 1
26 packet.hdr_done = 0
27 packet.payload_done = 1
28 packet.chk_ok = 0
29 packet.chk_fail = 0
30 packet.abort = 0
31 packet.valid_pkt = 0
32 packet.error_pkt = 0
33 packet.state = 3'b010 (010)
34 packet.IDLE = 3'b000 (000)
35 packet.HEADER = 3'b001 (001)
36 packet.PAYLOAD = 3'b010 (010)
37 packet.CHECKSUM = 3'b011 (011)
38 packet.DONE = 3'b100 (100)
39 packet.past_valid = 1'b0
40
41 Transition system state 1
42 -----
43 packet.clk = ?
44 packet.reset = 0

```

```
45 packet.start_pkt = 1
46 packet.hdr_done = 0
47 packet.payload_done = 0
48 packet.chk_ok = 0
49 packet.chk_fail = 0
50 packet.abort = 0
51 packet.valid_pkt = 0
52 packet.error_pkt = 0
53 packet.state = 3'b000 (000)
54 packet.IDLE = 3'b000 (000)
55 packet.HEADER = 3'b001 (001)
56 packet.PAYLOAD = 3'b010 (010)
57 packet.CHECKSUM = 3'b011 (011)
58 packet.DONE = 3'b100 (100)
59 packet.past_valid = 1'b1
60
61 Transition system state 2
62 -----
63 packet.clk = ?
64 packet.reset = 0
65 packet.start_pkt = 1
66 packet.hdr_done = 0
67 packet.payload_done = 0
68 packet.chk_ok = 0
69 packet.chk_fail = 1
70 packet.abort = 0
71 packet.valid_pkt = 0
72 packet.error_pkt = 0
73 packet.state = 3'b001 (001)
74 packet.IDLE = 3'b000 (000)
75 packet.HEADER = 3'b001 (001)
76 packet.PAYLOAD = 3'b010 (010)
77 packet.CHECKSUM = 3'b011 (011)
78 packet.DONE = 3'b100 (100)
79 packet.past_valid = 1'b1
80
81 Transition system state 3
82 -----
83 packet.clk = ?
84 packet.reset = 0
85 packet.start_pkt = 0
86 packet.hdr_done = 1
87 packet.payload_done = 0
88 packet.chk_ok = 0
89 packet.chk_fail = 0
90 packet.abort = 0
91 packet.valid_pkt = 0
92 packet.error_pkt = 0
93 packet.state = 3'b001 (001)
94 packet.IDLE = 3'b000 (000)
95 packet.HEADER = 3'b001 (001)
96 packet.PAYLOAD = 3'b010 (010)
97 packet.CHECKSUM = 3'b011 (011)
98 packet.DONE = 3'b100 (100)
99 packet.past_valid = 1'b1
100
101 Transition system state 4
102 -----
103 packet.clk = ?
104 packet.reset = 1
105 packet.start_pkt = 0
106 packet.hdr_done = 0
107 packet.payload_done = 0
```

```
108 packet.chk_ok = 1
109 packet.chk_fail = 0
110 packet.abort = 0
111 packet.valid_pkt = 0
112 packet.error_pkt = 0
113 packet.state = 3'b010 (010)
114 packet.IDLE = 3'b000 (000)
115 packet.HEADER = 3'b001 (001)
116 packet.PAYLOAD = 3'b010 (010)
117 packet.CHECKSUM = 3'b011 (011)
118 packet.DONE = 3'b100 (100)
119 packet.past_valid = 1'b1
120
121 Transition system state 5
122 -----
123 packet.clk = ?
124 packet.reset = 0
125 packet.start_pkt = 1
126 packet.hdr_done = 0
127 packet.payload_done = 0
128 packet.chk_ok = 0
129 packet.chk_fail = 0
130 packet.abort = 0
131 packet.valid_pkt = 0
132 packet.error_pkt = 0
133 packet.state = 3'b000 (000)
134 packet.IDLE = 3'b000 (000)
135 packet.HEADER = 3'b001 (001)
136 packet.PAYLOAD = 3'b010 (010)
137 packet.CHECKSUM = 3'b011 (011)
138 packet.DONE = 3'b100 (100)
139 packet.past_valid = 1'b1
140
141 Transition system state 6
142 -----
143 packet.clk = ?
144 packet.reset = 0
145 packet.start_pkt = 0
146 packet.hdr_done = 0
147 packet.payload_done = 0
148 packet.chk_ok = 1
149 packet.chk_fail = 0
150 packet.abort = 0
151 packet.valid_pkt = 0
152 packet.error_pkt = 0
153 packet.state = 3'b001 (001)
154 packet.IDLE = 3'b000 (000)
155 packet.HEADER = 3'b001 (001)
156 packet.PAYLOAD = 3'b010 (010)
157 packet.CHECKSUM = 3'b011 (011)
158 packet.DONE = 3'b100 (100)
159 packet.past_valid = 1'b1
160
161 Transition system state 7
162 -----
163 packet.clk = ?
164 packet.reset = 0
165 packet.start_pkt = 0
166 packet.hdr_done = 1
167 packet.payload_done = 0
168 packet.chk_ok = 0
169 packet.chk_fail = 0
170 packet.abort = 0
```

```

171 packet.valid_pkt = 0
172 packet.error_pkt = 0
173 packet.state = 3'b001 (001)
174 packet.IDLE = 3'b000 (000)
175 packet.HEADER = 3'b001 (001)
176 packet.PAYLOAD = 3'b010 (010)
177 packet.CHECKSUM = 3'b011 (011)
178 packet.DONE = 3'b100 (100)
179 packet.past_valid = 1'b1
180
181 Transition system state 8
182 -----
183 packet.clk = ?
184 packet.reset = 0
185 packet.start_pkt = 0
186 packet.hdr_done = 0
187 packet.payload_done = 0
188 packet.chk_ok = 1
189 packet.chk_fail = 0
190 packet.abort = 1
191 packet.valid_pkt = 0
192 packet.error_pkt = 0
193 packet.state = 3'b010 (010)
194 packet.IDLE = 3'b000 (000)
195 packet.HEADER = 3'b001 (001)
196 packet.PAYLOAD = 3'b010 (010)
197 packet.CHECKSUM = 3'b011 (011)
198 packet.DONE = 3'b100 (100)
199 packet.past_valid = 1'b1
200
201 Transition system state 9
202 -----
203 packet.clk = ?
204 packet.reset = 1
205 packet.start_pkt = 0
206 packet.hdr_done = 0
207 packet.payload_done = 0
208 packet.chk_ok = 0
209 packet.chk_fail = 0
210 packet.abort = 1
211 packet.valid_pkt = 0
212 packet.error_pkt = 0
213 packet.state = 3'b101 (101)
214 packet.IDLE = 3'b000 (000)
215 packet.HEADER = 3'b001 (001)
216 packet.PAYLOAD = 3'b010 (010)
217 packet.CHECKSUM = 3'b011 (011)
218 packet.DONE = 3'b100 (100)
219 packet.past_valid = 1'b1
220
221 [packet._state] always (disable iff (packet.reset) packet.state == packet.IDLE
    || packet.state == packet.HEADER || packet.state == packet.PAYLOAD || packet
    .state == packet.CHECKSUM || packet.state == packet.DONE): REFUTED
222 Counterexample:
223
224 Transition system state 0
225 -----
226 packet.clk = ?
227 packet.reset = 1
228 packet.start_pkt = 1
229 packet.hdr_done = 0
230 packet.payload_done = 1
231 packet.chk_ok = 0

```

```

232 packet.chk_fail = 0
233 packet.abort = 0
234 packet.valid_pkt = 0
235 packet.error_pkt = 0
236 packet.state = 3'b010 (010)
237 packet.IDLE = 3'b000 (000)
238 packet.HEADER = 3'b001 (001)
239 packet.PAYLOAD = 3'b010 (010)
240 packet.CHECKSUM = 3'b011 (011)
241 packet.DONE = 3'b100 (100)
242 packet.past_valid = 1'b0
243
244 Transition system state 1
245 -----
246 packet.clk = ?
247 packet.reset = 0
248 packet.start_pkt = 1
249 packet.hdr_done = 0
250 packet.payload_done = 0
251 packet.chk_ok = 0
252 packet.chk_fail = 0
253 packet.abort = 0
254 packet.valid_pkt = 0
255 packet.error_pkt = 0
256 packet.state = 3'b000 (000)
257 packet.IDLE = 3'b000 (000)
258 packet.HEADER = 3'b001 (001)
259 packet.PAYLOAD = 3'b010 (010)
260 packet.CHECKSUM = 3'b011 (011)
261 packet.DONE = 3'b100 (100)
262 packet.past_valid = 1'b1
263
264 Transition system state 2
265 -----
266 packet.clk = ?
267 packet.reset = 0
268 packet.start_pkt = 1
269 packet.hdr_done = 0
270 packet.payload_done = 0
271 packet.chk_ok = 0
272 packet.chk_fail = 1
273 packet.abort = 0
274 packet.valid_pkt = 0
275 packet.error_pkt = 0
276 packet.state = 3'b001 (001)
277 packet.IDLE = 3'b000 (000)
278 packet.HEADER = 3'b001 (001)
279 packet.PAYLOAD = 3'b010 (010)
280 packet.CHECKSUM = 3'b011 (011)
281 packet.DONE = 3'b100 (100)
282 packet.past_valid = 1'b1
283
284 Transition system state 3
285 -----
286 packet.clk = ?
287 packet.reset = 0
288 packet.start_pkt = 0
289 packet.hdr_done = 1
290 packet.payload_done = 0
291 packet.chk_ok = 0
292 packet.chk_fail = 0
293 packet.abort = 0
294 packet.valid_pkt = 0

```

```
295     packet.error_pkt = 0
296     packet.state = 3'b001 (001)
297     packet.IDLE = 3'b000 (000)
298     packet.HEADER = 3'b001 (001)
299     packet.PAYLOAD = 3'b010 (010)
300     packet.CHECKSUM = 3'b011 (011)
301     packet.DONE = 3'b100 (100)
302     packet.past_valid = 1'b1
303
304 Transition system state 4
305 -----
306     packet.clk = ?
307     packet.reset = 1
308     packet.start_pkt = 0
309     packet.hdr_done = 0
310     packet.payload_done = 0
311     packet.chk_ok = 1
312     packet.chk_fail = 0
313     packet.abort = 0
314     packet.valid_pkt = 0
315     packet.error_pkt = 0
316     packet.state = 3'b010 (010)
317     packet.IDLE = 3'b000 (000)
318     packet.HEADER = 3'b001 (001)
319     packet.PAYLOAD = 3'b010 (010)
320     packet.CHECKSUM = 3'b011 (011)
321     packet.DONE = 3'b100 (100)
322     packet.past_valid = 1'b1
323
324 Transition system state 5
325 -----
326     packet.clk = ?
327     packet.reset = 0
328     packet.start_pkt = 1
329     packet.hdr_done = 0
330     packet.payload_done = 0
331     packet.chk_ok = 0
332     packet.chk_fail = 0
333     packet.abort = 0
334     packet.valid_pkt = 0
335     packet.error_pkt = 0
336     packet.state = 3'b000 (000)
337     packet.IDLE = 3'b000 (000)
338     packet.HEADER = 3'b001 (001)
339     packet.PAYLOAD = 3'b010 (010)
340     packet.CHECKSUM = 3'b011 (011)
341     packet.DONE = 3'b100 (100)
342     packet.past_valid = 1'b1
343
344 Transition system state 6
345 -----
346     packet.clk = ?
347     packet.reset = 0
348     packet.start_pkt = 0
349     packet.hdr_done = 0
350     packet.payload_done = 0
351     packet.chk_ok = 1
352     packet.chk_fail = 0
353     packet.abort = 0
354     packet.valid_pkt = 0
355     packet.error_pkt = 0
356     packet.state = 3'b001 (001)
357     packet.IDLE = 3'b000 (000)
```

```

358 packet.HEADER = 3'b001 (001)
359 packet.PAYLOAD = 3'b010 (010)
360 packet.CHECKSUM = 3'b011 (011)
361 packet.DONE = 3'b100 (100)
362 packet.past_valid = 1'b1
363
364 Transition system state 7
365 -----
366 packet.clk = ?
367 packet.reset = 0
368 packet.start_pkt = 0
369 packet.hdr_done = 1
370 packet.payload_done = 0
371 packet.chk_ok = 0
372 packet.chk_fail = 0
373 packet.abort = 0
374 packet.valid_pkt = 0
375 packet.error_pkt = 0
376 packet.state = 3'b001 (001)
377 packet.IDLE = 3'b000 (000)
378 packet.HEADER = 3'b001 (001)
379 packet.PAYLOAD = 3'b010 (010)
380 packet.CHECKSUM = 3'b011 (011)
381 packet.DONE = 3'b100 (100)
382 packet.past_valid = 1'b1
383
384 Transition system state 8
385 -----
386 packet.clk = ?
387 packet.reset = 0
388 packet.start_pkt = 0
389 packet.hdr_done = 0
390 packet.payload_done = 0
391 packet.chk_ok = 1
392 packet.chk_fail = 0
393 packet.abort = 1
394 packet.valid_pkt = 0
395 packet.error_pkt = 0
396 packet.state = 3'b010 (010)
397 packet.IDLE = 3'b000 (000)
398 packet.HEADER = 3'b001 (001)
399 packet.PAYLOAD = 3'b010 (010)
400 packet.CHECKSUM = 3'b011 (011)
401 packet.DONE = 3'b100 (100)
402 packet.past_valid = 1'b1
403
404 Transition system state 9
405 -----
406 packet.clk = ?
407 packet.reset = 0
408 packet.start_pkt = 0
409 packet.hdr_done = 0
410 packet.payload_done = 0
411 packet.chk_ok = 1
412 packet.chk_fail = 0
413 packet.abort = 0
414 packet.valid_pkt = 0
415 packet.error_pkt = 0
416 packet.state = 3'b101 (101)
417 packet.IDLE = 3'b000 (000)
418 packet.HEADER = 3'b001 (001)
419 packet.PAYLOAD = 3'b010 (010)
420 packet.CHECKSUM = 3'b011 (011)

```

```

421 packet.DONE = 3'b100 (100)
422 packet.past_valid = 1'b1
423
424 [packet.idle_header] always (disable iff (packet.reset) packet.state == packet.
    IDLE && packet.start_pkt && !packet.abort | => packet.state == packet.HEADER)
    : PROVED up to bound 20
425 [packet.idle_idle] always (disable iff (packet.reset) packet.state == packet.
    IDLE && !packet.start_pkt && !packet.abort | => packet.state == packet.IDLE):
    PROVED up to bound 20
426 [packet.header_payload] always (disable iff (packet.reset) packet.state ==
    packet.HEADER && packet.hdr_done && !packet.abort | => packet.state == packet.
    PAYLOAD): PROVED up to bound 20
427 [packet.header_header] always (disable iff (packet.reset) packet.state ==
    packet.HEADER && !packet.hdr_done && !packet.abort | => packet.state ==
    packet.HEADER): PROVED up to bound 20
428 [packet.payload_checksum] always (disable iff (packet.reset) packet.state ==
    packet.PAYLOAD && packet.payload_done && !packet.abort | => packet.state ==
    packet.CHECKSUM): PROVED up to bound 20
429 [packet.payload_payload] always (disable iff (packet.reset) packet.state ==
    packet.PAYLOAD && !packet.payload_done && !packet.abort | => packet.state ==
    packet.PAYLOAD): PROVED up to bound 20
430 [packet.checksum_done] always (disable iff (packet.reset) packet.state ==
    packet.CHECKSUM && packet.chk_ok && !packet.abort | => packet.state == packet.
    DONE && packet.valid_pkt): PROVED up to bound 20
431 [packet.checksum_idle] always (disable iff (packet.reset) packet.state ==
    packet.CHECKSUM && packet.chk_fail && !packet.abort | => packet.state ==
    packet.IDLE && packet.error_pkt): PROVED up to bound 20
432 [packet.checksum_checksum] always (disable iff (packet.reset) packet.state ==
    packet.CHECKSUM && !packet.chk_ok && !packet.chk_fail && !packet.abort | =>
    packet.state == packet.CHECKSUM && !packet.valid_pkt && !packet.error_pkt):
    PROVED up to bound 20
433 [packet.done_idle] always (disable iff (packet.reset) packet.state == packet.
    DONE && !packet.abort | => packet.state == packet.IDLE): PROVED up to bound
    20
434 [packet.pulse_error_pkt] always (disable iff (packet.reset) packet.past_valid
    && packet.error_pkt | -> !$past(packet.error_pkt)): PROVED up to bound 20
435 [packet.safe_error_pkt] always (disable iff (packet.reset) packet.past_valid &&
    packet.error_pkt | -> $past(packet.state) == packet.CHECKSUM && $past(packet.
    chk_fail) && !$past(packet.abort)): PROVED up to bound 20
436 [packet.pulse_valid_pkt] always (disable iff (packet.reset) packet.past_valid
    && packet.valid_pkt | -> !$past(packet.valid_pkt)): PROVED up to bound 20
437 [packet.safe_valid_pkt] always (disable iff (packet.reset) packet.past_valid &&
    packet.valid_pkt | -> $past(packet.state) == packet.CHECKSUM && $past(packet.
    chk_ok) && !$past(packet.abort)): PROVED up to bound 20

```

